

ÔN THI GIỮA KỲ

Môn học: Nguyên lý ngôn ngữ lập trình

Chương 1: GIỚI THIỆU

Câu 01. Một lập trình viên viết chương trình bằng ngôn ngữ Z. Khi chạy, anh ta nhận thấy chương trình bắt đầu chậm nhưng dần trở nên nhanh hơn sau vài lần lặp lại mà không cần biên dịch lại. Dựa trên hiện tượng này, hãy xác định phương pháp dịch có thể đã được sử dụng và giải thích cơ chế hoạt động của nó.

Câu 02. Một trình dịch lại mất 3 giây để biên dịch mã nguồn thành bytecode, sau đó thực thi với tốc độ 0,8 giây mỗi dòng lệnh. Trong khi đó, một trình thông dịch thuần túy mất 2,5 giây để thực thi mỗi dòng lệnh. Hãy xác định số dòng lệnh tối thiểu để trình dịch lại có tổng thời gian nhỏ hơn trình thông dịch thuần túy.

Câu 03. Một chương trình có hai lựa chọn để thực thi:

- Dùng biên dịch truyền thống: Mất 7 giây để biên dịch, nhưng sau đó chạy với tốc độ 0,5 giây mỗi dòng lệnh.
- Dùng trình dịch tức thời (JIT): Không mất thời gian biên dịch ban đầu, nhưng mỗi dòng mất 1,2 giây khi chạy lần đầu và 0,4 giây khi chạy lại.

Với chương trình có N dòng lệnh, trong đó 40% số dòng được thực thi 2 lần, hãy tìm giá trị N tối thiểu để trình dịch tức thời (JIT) có tổng thời gian thực thi nhanh hơn phương pháp biên dịch truyền thống.

Câu 04. Công cụ nào có nhiệm vụ kết hợp các tệp đối tượng (object files) lại với nhau và tạo ra một tệp thực thi?

Câu 05. Điền vào chỗ trống: "_____ đảm bảo rằng tất cả các chương trình và thư viện cần thiết được nạp vào bộ nhớ trước khi thực thi."

Chương 2: PHÂN TÍCH TỪ VỰNG

Câu 01. Cho các loại lỗi sau:

- Một biến chưa được khai báo nhưng được sử dụng
- Một từ khóa được sử dụng làm tên biến
- Một dấu @ xuất hiện trong mã nguồn C
- Một dấu ngoặc đóng bị thiếu
- Một số nguyên có dấu chấm thập phân.

Có bao nhiêu lỗi trong danh sách trên mà bộ phân tích từ vựng có thể phát hiện?

Câu 02. Cho đoạn mã sau được viết bằng C++:

```
int lvar = 10; float num = 12.; char ch = '$';
```

Có bao nhiêu lỗi mà bộ phân tích từ vựng (lexical analyzer) có thể phát hiện trong đoạn mã trên?

Câu 03. Trong một tập luật của ANTLR4, bạn có quy tắc lexer như sau: `STRING : ''' .*? ''' ;`. Quy tắc này cho phép nhận diện chuỗi nằm giữa hai dấu " nhưng không thay đổi nội dung bên trong. Bạn muốn thay đổi action để mọi chuỗi "hello" được thay bằng "STRING CONSTANT" khi token được xử lý. Hãy mô tả cách thực hiện điều này và giải thích cách thức hoạt động của lexer trong ANTLR4 để đạt được kết quả mong muốn.

Câu 04. Một bộ phân tích từ vựng sử dụng quy tắc lexer như sau:

MULTI_LINE_COMMENT : '/' '*' . '*?' '/' -> skip;

SINGLE_LINE_COMMENT : '/' '/' '[' '*' -> skip;

Tuy nhiên, khi biên dịch một tập tin mã nguồn chứa nhiều dòng chú thích, chương trình đôi khi gặp lỗi hoặc hoạt động không như mong muốn. Hãy xác định các tình huống có thể gây ra lỗi với quy tắc trên và đề xuất một cách viết quy tắc tốt hơn để tránh lỗi trong ANTLR4.

Câu 05. Cho biểu thức chính quy $(0|1)^*101$, hãy thiết kế một bộ tự động hữu hạn xác định (DFA) để nhận diện tập hợp chuỗi thỏa mãn biểu thức này. Xác định số trạng thái tối thiểu cần có trong DFA

Câu 06. Một chuỗi nhị phân được gọi là "chuỗi đặc biệt" nếu nó chứa đúng hai lần xuất hiện của dãy con "101" và không chứa "110".

- Hãy viết một biểu thức chính quy mô tả tập hợp chuỗi "đặc biệt" này.
- Xây dựng một bộ DFA tối thiểu để nhận diện các chuỗi "đặc biệt".

Chương 3: PHÂN TÍCH NGỮ PHÁP

Câu 01. Cho các loại lỗi lập trình sau:

- Một biến được sử dụng mà chưa khai báo.
- Một hàm được gọi với sai số lượng tham số.
- Một phép toán chia một số cho 0.
- Một vòng lặp vô hạn mà không có điều kiện dừng.
- Một cú pháp sai do quên dấu ; trong C++.

Hãy phân loại các lỗi trên thành lỗi cú pháp, lỗi ngữ nghĩa, hoặc lỗi runtime (thời gian chạy) và giải thích lý do cho từng loại lỗi.

Câu 02. Trong một ngôn ngữ lập trình, một số toán tử có thể được định nghĩa theo văn phạm sau:

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

- Giải thích tại sao văn phạm này bị nhập nhằng.
- Hãy đề xuất cách chỉnh sửa văn phạm để đảm bảo độ ưu tiên toán tử (* có độ ưu tiên cao hơn +), nhưng vẫn giữ đúng ngữ nghĩa của ngôn ngữ.

Câu 03. Cho văn phạm phi ngữ cảnh sau: $S \rightarrow aSbS \mid \epsilon$

- Chứng minh rằng văn phạm này sinh ra tất cả các chuỗi có số lượng ký tự a bằng số lượng ký tự b, nhưng không nhất thiết phải ở dạng $a^n b^n$.
- Xây dựng **quy nạp toán học** để chứng minh rằng mọi chuỗi sinh ra từ văn phạm trên có thể được biểu diễn dưới dạng **chuỗi Dyck**, tức là một chuỗi dấu ngoặc hợp lệ trong đó "a" đóng vai trò dấu mở và "b" đóng vai trò dấu đóng.
- Chứng minh rằng văn phạm trên **không nhập nhằng** bằng cách chỉ ra rằng mọi chuỗi hợp lệ có **duy nhất một cây phân tích cú pháp**.

Câu 04. Xét hai văn phạm phi ngữ cảnh sau:

Văn phạm 1

$$S \rightarrow aSbc \mid abc$$

Văn phạm 2

$$S \rightarrow abSc \mid abc$$

- (a) Hai văn phạm này có sinh ra cùng một ngôn ngữ hay không? Giải thích bằng cách so sánh tập hợp chuỗi mà mỗi văn phạm có thể tạo ra.
- (b) Nếu có sự khác biệt, hãy điều chỉnh một trong hai văn phạm để cả hai cùng sinh ra một ngôn ngữ duy nhất.

Chương 4: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VÀ LẬP TRÌNH HÀM

Câu 01. Trong lập trình hướng đối tượng, ràng buộc động (Dynamic Binding) và đa hình (Polymorphism) có mối liên hệ mật thiết với nhau.

- (a) Hãy giải thích vai trò của ràng buộc động trong việc hỗ trợ tính đa hình.
- (b) Nếu một ngôn ngữ lập trình không hỗ trợ ràng buộc động, thì cách duy nhất để hỗ trợ đa hình là gì? Đưa ra ví dụ minh họa bằng mã giả hoặc một ngôn ngữ lập trình cụ thể.

Câu 02. Một số ngôn ngữ như C++ cho phép cả ràng buộc tĩnh và ràng buộc động, trong khi các ngôn ngữ như Python thực hiện ràng buộc động hoàn toàn.

- (a) Hãy so sánh cách hoạt động của ràng buộc động trong Python với cách hoạt động của từ khóa virtual trong C++.
- (b) Điều gì sẽ xảy ra nếu một phương thức ảo (virtual method) trong C++ không được khai báo bằng từ khóa virtual? Nó có ảnh hưởng đến tính đa hình không? Giải thích bằng ví dụ.

Câu 03. C++ có các mức độ truy cập (public, protected, private), trong khi Python không có từ khóa kiểm soát truy cập chính thức mà chỉ dựa vào quy ước (_protected, __private).

- (a) Tại sao Python không bắt buộc các mức độ truy cập như C++?
- (b) Hãy viết hai đoạn mã trong C++ và Python để minh họa cách truy cập thuộc tính private từ bên ngoài lớp. Python có thực sự ngăn chặn được truy cập như C++ không?

Câu 04. Xét đoạn mã Python sau và :

```
class A:
    def __init__(self):
        print("A", end=" ")

class B(A):
    def __init__(self):
        print("B", end=" ")
        super().__init__()

class C(A):
    def __init__(self):
        print("C", end=" ")
        super().__init__()
```

```
class D(B, C):
    def __init__(self):
        print("D", end=" ")
        super().__init__()
```

D()

Trả lời các câu hỏi:

- Kết quả của chương trình trên là gì?
- Giải thích thứ tự gọi các phương thức `__init__()` của các lớp theo MRO.
- Nếu thay `super().__init__()` trong B và C bằng `A.__init__(self)`, kết quả có thay đổi không? Nếu có, hãy giải thích.

Câu 05.

Xét đoạn mã sau:

```
x = (1, 2, 3)
y = x
x += (4, 5)
```

- Biến `y` có bị thay đổi sau khi thực hiện lệnh `x += (4, 5)` không?
- Nếu `x` là một danh sách thay vì tuple (`x = [1, 2, 3]`), kết quả có khác không? Giải thích.
- Dựa vào câu trên, hãy mô tả sự khác biệt giữa **thay đổi nội dung của một đối tượng mutable** và **gán lại một biến immutable**.

Câu 06. Cho đoạn mã sau:

```
a = (1, 2, [3, 4])
a[2].append(5)
print(a)
```

- Kết quả của chương trình trên là gì? Giải thích tại sao chúng ta có thể thay đổi nội dung của `a`, mặc dù `a` là một tuple.
- Làm thế nào để tạo một tuple hoàn toàn immutable, ngay cả khi nó chứa danh sách?

Câu 07. Currying và function composition là hai khái niệm quan trọng trong lập trình hàm.

- Hãy giải thích sự khác biệt giữa **currying** và **function composition** trong Python.
- Viết một đoạn mã Python sử dụng **currying function** để tạo một hàm `add_prefix(prefix)` thêm tiền tố vào một chuỗi, và sau đó sử dụng **function composition** để kết hợp `add_prefix("Mr. ")` với một hàm `to_uppercase()` để tạo ra hàm `shout_with_prefix(name)`.
- Khi nào nên sử dụng **currying** thay vì **function composition**, và ngược lại?

Câu 08. Python cho phép sử dụng **nhiều decorators** trên một hàm.

- Viết một đoạn mã trong đó một hàm được trang bị hai decorators:
 - `uppercase`: Chuyển đổi output thành chữ in hoa.
 - `times(n)`: Lặp lại việc gọi hàm `n` lần.

- (b) Khi thực thi hàm, kết quả sẽ bị ảnh hưởng như thế nào bởi thứ tự khai báo decorators?

Chương 5: SINH CÂY CÚ PHÁP TRỪU TƯỢNG

Câu 01. Trong quá trình biên dịch, **Parse Tree** và **AST** đóng vai trò quan trọng trong phân tích cú pháp và tạo mã trung gian.

- (a) Hãy giải thích sự khác biệt chi tiết giữa **Parse Tree** và **AST**, tập trung vào cấu trúc, mục đích và mức độ trừu tượng.
- (b) Viết một ví dụ với biểu thức toán học $(3 + 4) * 5$, vẽ **Parse Tree** và **AST** tương ứng.
- (c) Tại sao trình biên dịch thường sử dụng **AST** thay vì **Parse Tree** trong các giai đoạn sau của quá trình biên dịch?